

Democratic and Popular Republic of Algeria
Ministry of Higher Education and Scientific Research

University Abderrahmane Mira of Bejaia



Faculty of Exact Sciences
Department of Computer Science

Project Report

Module: Software Engineering
Medical Laboratory Management System

Rezouali Imane
Groupe : B3
Academic Year: 2025/2026

Introduction

This project entails the design and implementation of **MediLab**, a comprehensive Medical Lab Management System. This desktop application is engineered to orchestrate the entire lifecycle of a medical test through a centralized, state-driven workflow.

The primary objectives of the **MediLab** project are:

- **Operational Synchronization:** To bridge the operational gap between the front office (Secretary) and the back office (Technician and Financial Manager). The system ensures that the Secretary can only deliver results or invoices once they have been explicitly validated by the respective back office specialists.
- **Process Automation:** To digitize the "Sandwich Workflow," where the process is initiated by the Secretary (Appointment/Intake), processed by the Technician (Analysis), and concluded by the Secretary (Final Delivery), thereby eliminating manual handovers and reducing human error.
- **Data Integrity and Security:** To enforce strict validation rules ensuring that clinical data is immutable after validation—and to protect sensitive patient information through a rigorous security model.

Technical Architecture

From a software engineering perspective, **MediLab** is built using the **Java** programming language, adhering to the industry standard **Three Tier Architecture**:

1. **Presentation Tier:** Handles user interaction via a responsive Graphical User Interface (GUI) in Java Swing.
2. **Application Tier (also called Business Logic):** Encapsulates the core rules of the laboratory, such as test price calculations and status transitions.
3. **Data Access Tier (DAO):** Manages persistent storage and interaction with the relational database.

Furthermore, the system integrates a robust **Role Based Access Control (RBAC)** mechanism. This hierarchy distinguishes between the **Super Admin** (System Oversight), **Admin** (works like HR Management), **Secretary** (Logistics and Delivery), **Technician** (Clinical Processing), and **Financial Manager** (Billing Validation), ensuring that each actor operates strictly within their authorized domain.

Requirements Analysis

To ensure the proposed solution addresses real world challenges, an operational audit was conducted at local medical laboratories and Medical facilities. Specifically, field observations were carried out at:

- **Laboratoire d'Analyses Médicales Dr.s Oukachbi Epse IDRIR**
- **Dr.Saidani L, Hépatogastro-Entérologue**

We observed the daily workflow of staff at these locations to understand the operational logic and identify the core processes that require digitization. These observations provided the foundational blueprint for the **MediLab** architecture.

Analysis of the Existing System (Design Inspiration)

Rather than reinventing the laboratory workflow, our analysis of the existing manual systems highlighted effective patterns that inspired the design of our application:

We observed that laboratories naturally operate in two distinct zones: the Reception (Administrative) and the Lab (Technical). This physical separation inspired our software's **Distributed Architecture**, where the Secretary and Technician have distinct, isolated interfaces that communicate via a central server.

In successful laboratories, efficiency relies on strict specialization Secretaries handle logistics while Technicians handle diagnostics. This clear division of labor directly inspired our **Role Based Access Control (RBAC)** model, ensuring that digital privileges mirror these real world professional boundaries.

Functional Requirements

Building upon the "Physical State Machine" observed in the field, the **MediLab** system is designed around a strict "**Sandwich Workflow**" that digitally bridges the Front and Back offices.

Actor-Specific Requirements

The system must enforce the separation of duties observed during the analysis.

- **The Secretary (Front Office , Entry/Exit):**
 - Must be able to register patients and schedule appointments digitally.

- *Digital Handover*: Replicating the physical handover, the Secretary acts as the delivery interface but cannot modify clinical results. The system must notify(or show to) the Secretary when a result status changes to **VALIDATED**.
- **The Technician (Back Office , Processing):**
 - Needs a digital "worklist" to view pending appointments, replacing the physical inbox.
 - Must be the only actor authorized to input test values and trigger the validation status, preserving the clinical authority observed in the field.
- **The Financial Manager (Financial Control):**
 - Inspired by the need for financial tracking, this actor validates payments independently.
 - The system must block the Secretary from printing an invoice until the Financial Manager has flagged it as **PAID**.

Operational Workflow Requirements

The application digitizes the lifecycle of a medical test through three distinct states to ensure synchronization:

1. **Initiation Phase:** The Patient arrives. The Secretary creates an **Appointment**. At this stage, the status is **SCHEDULED**.
2. **Processing Phase:** The Technician converts the appointment into a **TestOrder**. Clinical work begins. Simultaneously, the Financial Manager processes the **Invoice**.
3. **Delivery Phase:** The system automatically unlocks the final report for printing by the Secretary only when:

$$ResultStatus == VALIDATED \quad \text{AND} \quad PaymentStatus == PAID$$

Non-Functional Requirements

- **Data Integrity:** The system must prevent "Orphaned Orders" (Results without a patient).
- **Simplicity:** Given the fast paced environment observed, the UI for the Secretary must be minimal and efficient
- **Security:** Sensitive medical data must be accessible only to authenticated users with the correct role privileges.



Database Design

To convert the Class Diagram into a relational schema, the following rules were applied:

1. **Inheritance Mapping (Single Table):** The `Employee` hierarchy (Secretary, Technician, etc.) is mapped to a single table with a `role` discriminator column. This avoids unnecessary table joins during login authentication.
2. **Association Mapping:**
 - **1-to-N Associations:** A Foreign Key (FK) is added to the "Many" side (e.g., `patientId` inside `TestOrder`).
 - **1-to-1 Associations:** The relationship between `TestOrder` and `Invoice` implies that one order generates exactly one invoice. The Foreign Key is placed in the `Invoice` table to maintain data integrity.

Relational Model (MLD)

The following schema defines the database tables.

- **Employee** (employeeId, firstName, lastName, CIN, phone, email, address, hireDate, role, username, password)
- **Patient** (patientId, firstName, lastName, dateOfBirth, gender, phone, email, address, registrationDate)
- **TestCategory** (categoryId, name, description)
- **TestType** (testTypeId, name, description, price, normalRange, #categoryId)
- **Appointment** (appointmentId, date, reason, status, #patientId, #secretaryId)
- **TestOrder** (orderId, dateOrdered, status, #patientId, #technicianId)
- **TestResult** (resultId, value, interpretation, resultDate, isValidated, #orderId, #testTypeId, #technicianId)
- **Invoice** (invoiceId, date, totalAmount, paymentStatus, #orderId, #patientId)
- **Payment** (paymentId, amount, date, method, #invoiceId, #financialManagerId)

Entity-Relationship Diagram (ERD)

The ERD below visually represents the MLD. It highlights the relationships between the actors (via the Employee table) and the operational entities.

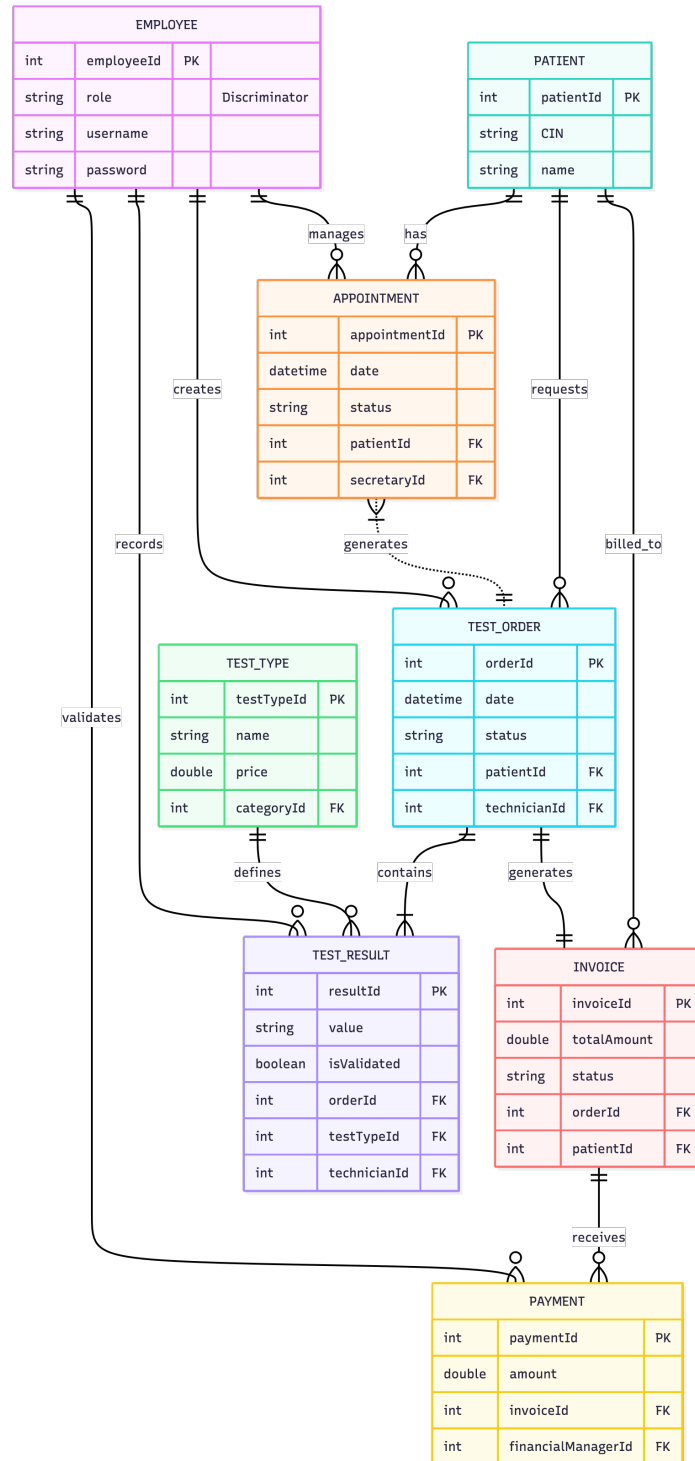


Figure 2: Entity-Relationship Diagram (ERD) of MediLab

Domain Diagram

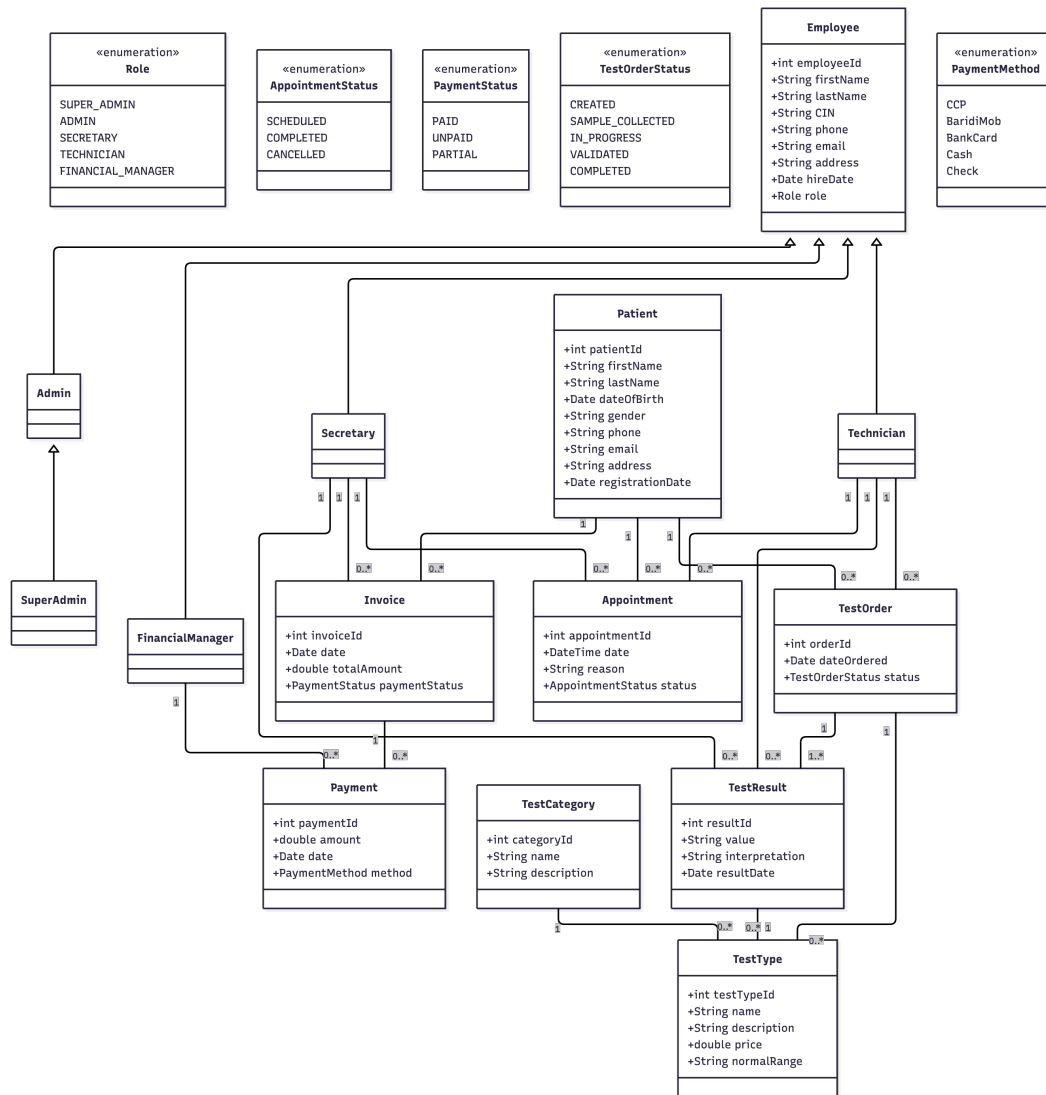


Figure 3: Domain Diagram of MediLab

Database Implementation (SQL Script)

The following SQL script implements the logical schema defined in the Relational Model. It creates the database `medical_lab_db` and defines all tables with appropriate Primary and Foreign Keys.

```
1  -- 1. DB creation
2  CREATE DATABASE IF NOT EXISTS medical_lab_db;
3  USE medical_lab_db;
4
5  -- 2. Employees Table
6  CREATE TABLE Employee (
7      employeeId INT AUTO_INCREMENT PRIMARY KEY,
8      firstName VARCHAR(50) NOT NULL,
9      lastName VARCHAR(50) NOT NULL,
10     username VARCHAR(50) UNIQUE NOT NULL,
11     password VARCHAR(255) NOT NULL,
12     role ENUM('SUPER_ADMIN', 'ADMIN', 'SECRETARY',
13              'TECHNICIAN', 'FINANCIAL_MANAGER') NOT NULL,
14     hireDate DATE,
15     phone VARCHAR(20),
16     address VARCHAR(255) NOT NULL DEFAULT '',
17     email VARCHAR(100)
18 );
19
20 -- 3. Patient Table
21 CREATE TABLE Patient (
22     patientId INT AUTO_INCREMENT PRIMARY KEY,
23     firstName VARCHAR(50) NOT NULL,
24     lastName VARCHAR(50) NOT NULL,
25     dateOfBirth DATE NOT NULL,
26     gender ENUM('Male', 'Female') NOT NULL,
27     phone VARCHAR(20),
28     email VARCHAR(100),
29     address VARCHAR(255),
30     registrationDate DATETIME DEFAULT CURRENT_TIMESTAMP
31 );
32
33 -- 4. Test Configuration Tables
34 CREATE TABLE TestCategory (
35     categoryId INT AUTO_INCREMENT PRIMARY KEY,
36     name VARCHAR(50) NOT NULL,
37     description TEXT
38 );
39
```

```

40 CREATE TABLE TestType (
41     testTypeId INT AUTO_INCREMENT PRIMARY KEY,
42     categoryId INT NOT NULL,
43     name VARCHAR(100) NOT NULL,
44     description TEXT,
45     price DECIMAL(10, 2) NOT NULL,
46     normalRange VARCHAR(100),
47     FOREIGN KEY (categoryId) REFERENCES TestCategory(categoryId)
48 );
49
50 -- 5. Appointment Table
51 CREATE TABLE Appointment (
52     appointmentId INT AUTO_INCREMENT PRIMARY KEY,
53     patientId INT NOT NULL,
54     secretaryId INT NOT NULL,
55     date DATETIME NOT NULL,
56     reason TEXT,
57     status ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED') DEFAULT '
SCHEDULED',
58     FOREIGN KEY (patientId) REFERENCES Patient(patientId),
59     FOREIGN KEY (secretaryId) REFERENCES Employee(employeeId)
60 );
61
62 -- 6. Test Order Table
63 CREATE TABLE TestOrder (
64     orderId INT AUTO_INCREMENT PRIMARY KEY,
65     patientId INT NOT NULL,
66     technicianId INT,
67     dateOrdered DATETIME DEFAULT CURRENT_TIMESTAMP,
68     status ENUM('CREATED', 'SAMPLE_COLLECTED', 'IN_PROGRESS',
69                 'VALIDATED', 'COMPLETED') DEFAULT 'CREATED',
70     FOREIGN KEY (patientId) REFERENCES Patient(patientId),
71     FOREIGN KEY (technicianId) REFERENCES Employee(employeeId)
72 );
73
74 -- 7. Test Result Table
75 CREATE TABLE TestResult (
76     resultId INT AUTO_INCREMENT PRIMARY KEY,
77     orderId INT NOT NULL,
78     testTypeId INT NOT NULL,
79     technicianId INT,
80     value VARCHAR(255),
81     interpretation TEXT,
82     isValidated BOOLEAN DEFAULT FALSE,
83     resultDate DATETIME,
84     FOREIGN KEY (orderId) REFERENCES TestOrder(orderId),
85     FOREIGN KEY (testTypeId) REFERENCES TestType(testTypeId),

```

```

86     FOREIGN KEY (technicianId) REFERENCES Employee(employeeId)
87 );
88
89 -- 8. Invoice Table
90 CREATE TABLE Invoice (
91     invoiceId INT AUTO_INCREMENT PRIMARY KEY,
92     orderId INT UNIQUE NOT NULL,
93     patientId INT NOT NULL,
94     date DATETIME DEFAULT CURRENT_TIMESTAMP,
95     totalAmount DECIMAL(10, 2) NOT NULL,
96     paymentStatus ENUM('PAID', 'UNPAID', 'PARTIAL') DEFAULT 'UNPAID',
97     FOREIGN KEY (orderId) REFERENCES TestOrder(orderId),
98     FOREIGN KEY (patientId) REFERENCES Patient(patientId)
99 );
100
101 -- 9. Payment Table
102 CREATE TABLE Payment (
103     paymentId INT AUTO_INCREMENT PRIMARY KEY,
104     invoiceId INT NOT NULL,
105     financialManagerId INT,
106     amount DECIMAL(10, 2) NOT NULL,
107     date DATETIME DEFAULT CURRENT_TIMESTAMP,
108     method ENUM('CCP', 'BaridiMob', 'BankCard', 'Cash', 'Check') NOT
109     NULL,
110     FOREIGN KEY (invoiceId) REFERENCES Invoice(invoiceId),
111     FOREIGN KEY (financialManagerId) REFERENCES Employee(employeeId)
112 );

```

Listing 1: MediLab Database Creation Script

Conclusion

The realization of the **MediLab** Medical Lab Management System constitutes a significant technical advancement in the digitization of diagnostic healthcare operations. By engineering a centralized platform based on the Three Tier Architecture, the project successfully decouples the presentation layer from complex business logic and data persistence, ensuring high maintainability and scalability. A core technical achievement is the implementation of a state driven orchestration model which strictly enforces separation of concerns between the administrative and clinical domains via Role-Based Access Control (RBAC). This architecture not only mitigates concurrency issues and data redundancy through a normalized relational schema but also guarantees the integrity of clinical reports and the atomicity of financial transactions.

Looking forward, the system's modular design provides a robust, API-ready foundation for interoperability extensions. Future iterations envision the integration of HL7 standards for direct middleware communication with auto analyzers, the deployment of a secure RESTful patient portal, and the implementation of asynchronous notification services. Consequently, MediLab transcends basic information management to become a resilient enterprise resource planning tool tailored for the rigorous technical demands of modern laboratory environments.